

Tâche D.2

groupe 32

April 2025

1 Expérience 1

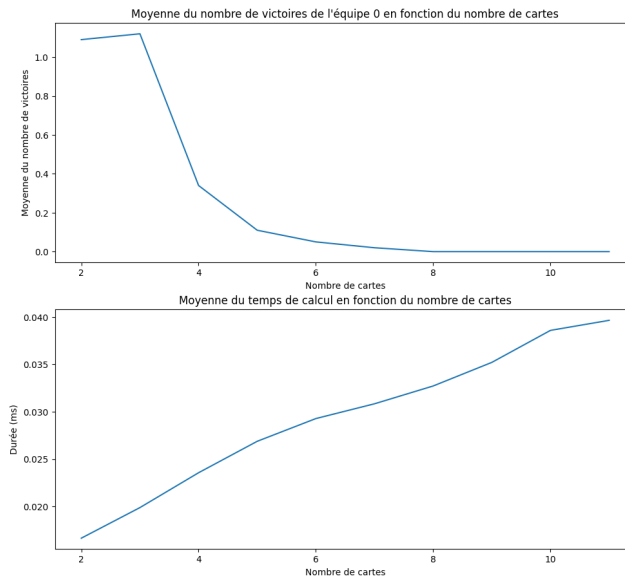
A travers cette expérience, j'ai fait varier le nombre de cartes dans le main du joueur. Pour cette expérience, les paramètres sont les suivants :

- nombre d'équipes : 2
- nombre de joueuses par équipe : 2
- nombre de tours par partie : 3
- nombre de cartes par joueuse : entre 2 et 11

Description des fonctions de mon algorithme :

- `sum_played_cards_1(board b, int team_id)`
Cette fonction compte la somme totale des cartes jouées par une équipe en comptant individuellement la somme des cartes jouées par chaque joueuse de l'équipe.
- `apply_points_1(board b, player players[], int s0, int s1)`
Cette fonction commence par comparer le score de l'équipe 0 et le score de l'équipe 1 afin de déterminer laquelle est gagnante et laquelle est perdante. Par la suite, pour chaque joueuse, elle regarde si l'équipe est gagnante ou perdante, et elle compare avec le pari de la joueuse. Si la joueuse est gagnante et que son ardoise vaut 1 alors son équipe a un point supplémentaire. A l'inverse, si elle est perdante et que son ardoise vaut 0 alors son équipe a un point supplémentaire également.
- `free_all_cards_1(player p)` Cette fonction libère toutes les cartes de la main d'une joueuse (en les enlevant de la main et en libérant la mémoire).
- `free_all_card_1(player p)` Cette fonction libère toutes les cartes jouées par la joueuse p (en les supprimant et en libérant la mémoire).
- `partie_1(int n)`
Cette fonction simule une partie complète entre les deux équipes (chaque joueuse a n cartes en main). Elle commence par initialiser diverses éléments. Elle crée un plateau, crée 2 équipes avec 2 joueuses chacune et elle distribue n cartes (de valeurs 1 ou 2 aléatoires) à chaque joueuse.
Par la suite, la joueuse 0 de l'équipe 0 utilise la méthode probabiliste tandis que les autres joueuses utilisent la stratégie agressive. Chaque joueuse joue une carte.
On calcule les scores, on nettoie en libérant la mémoire puis on retourne 1 si l'équipe 0 gagne, 0 sinon.
- `main()`
Pour finir le main permet de relier les différentes fonctions.
Il permet de simuler 100 parties avec à chaque fois 10 tours (chaque tour correspond à un nombre différent de cartes).

Résultats attendus :



2 Expérience 2

Dans cette expérience, j'ai fait varier le nombre de tours dans une partie. Les paramètres sont les suivants :

- nombre d'équipes : 2
- nombre de joueuses par équipe : 2
- nombre de tours par partie : entre 3 et 50
- nombre de cartes par joueuse : 5

Fonctions de mon algorithme

Les fonctions de mon algorithme sont identiques, mis-à-part dans :

- `partie_2(int n)`

Cette fonction prend cette fois-ci en paramètre le nombre de cartes par joueur. Le nombre de tours devient constant tandis que le nombre de cartes vaut n .

- `main()`

Cette fois-ci on simule le jeu en faisant varier le nombre de parties et non le nombre de cartes.

Résultats attendus :

